Indian Institute of Technology Kharagpur
Department of Computer Science and Engineering

CS60075: Natural Language Processing
Assignment II – Project Report

Do LLMs Know When They're Making Things Up?

Project 7: Hallucination Detection
Dataset: HaluEval (QA subset)

Student: Thilok
Roll number: 23CS30073
LLM used: `google/gemma-2-2b-it`
Submission date: November 8, 2026

Tasks covered. Task 1 – probability-based detection (log-probability and entropy); **Task 2** – SelfCheckGPT reproduction (BERTScore and NLI over N=20 resamples); **Task 3** – stacked ensemble detector (LR / RF / GBT) with bootstrap confidence intervals.

Abstract

A language model hallucinates when it produces fluent but factually wrong text. This report asks whether a model's own signals can reveal that an answer is hallucinated. Using the QA subset of HaluEval and the open instruction-tuned model Gemma-2-2B-it, we build and compare three detectors: (i) a *probability-based* detector from token log-probabilities and predictive entropy under forced decoding (Task 1); (ii) a reproduction of *SelfCheckGPT*, which measures consistency between an answer and N=20 stochastic resamples via BERTScore and NLI (Task 2); and (iii) a *stacked meta-classifier* over both feature families (Task 3). We evaluate with AUC-PR on a fixed, question-level 80/20 split (160 test instances) and add bootstrap confidence intervals.

The specified baselines behave unexpectedly: mean log-probability and perplexity are *below chance* (AUC-PR 0.384), and mean NLI contradiction is also below chance (0.455). Entropy-based scores work well (mean entropy 0.857, max entropy 0.932), and SelfCheckGPT works only after adaptation: prepending the question lifts BERTScore from 0.512 to 0.712, and inverting the NLI signal (using $1 - P(\text{entailment})$) reaches 0.700. The Task 3 logistic-regression ensemble reaches **0.9985** AUC-PR (95% CI 0.9951–1.0000). However, a one-feature baseline, the answer's token count, already reaches 0.980, so most of the achievable performance on this benchmark is explained by a length artifact rather than by genuine model self-knowledge. We analyse this finding in detail and discuss how to evaluate hallucination detectors more fairly.

Contents

Abstract **i**

1 Introduction **1**

1.1 Problem statement 1

1.2 Contributions 1

1.3 Summary of key results 2

1.4 Report organisation 2

2 Dataset, Models and Evaluation Protocol **2**

2.1 HaluEval (QA subset) 2

2.2 Sampling and splits 3

2.3 Models 3

2.4 System overview 4

2.5 Evaluation metrics 4

2.6 Reproducibility 4

3 Task 1 – Probability-Based Hallucination Detection **4**

3.1 Objective 5
3.2 Method 5
3.3 Results 6
3.4 Analysis 7

4 Task 2 – Paper Reproduction: SelfCheckGPT 7
4.1 Objective 8
4.2 Idea 8
4.3 Resampling protocol 8
4.4 Scoring functions 8
4.5 Results 9
4.6 Comparison with the original paper 10
4.7 Analysis 11

5 Task 3 – Combine and Improve: Stacked Ensemble Detector 12
5.1 Objective 12
5.2 Feature table 12
5.3 Models and training protocol 12
5.4 Results 13
5.5 Task 1 vs. Task 2 vs. Task 3 14
5.6 Analysis 14

6 Cross-Task Comparison 16

7 Conclusion 16

List of Tables

1 Headline results on the 160-instance test split (AUC-PR; random = 0.5). 2
2 Dataset partitions (seed 42). 3
3 Task 1 features and the score orientation used for evaluation. 5
4 Task 1 test-set results (160 instances, 80 pairs). Best value per column in bold; shaded rows are below chance on AUC-PR. 6
5 Resampling configuration. 8
6 The eight Task 2 features (all oriented so that higher means more likely hallucinated). 9
7 Task 2 results on the 160-instance test split. Rows shaded red are below chance on AUC-PR; bold marks the two best. 9
8 AUC-PR of selected Task 2 variants on all three evaluation sets. 10
9 Feature families used by the meta-classifiers. 12
10 Meta-classifiers and the hyperparameter grids searched. 12
11 Cross-validated (5-fold group, on train) and held-out test AUC-PR. 13
12 Final comparison on the 160-instance test split with 95% cluster-bootstrap confidence intervals ($B = 2000$ over 80 questions). 13
13 Required three-way comparison on the same 160 test instances. 14
14 Which families of signal work on HaluEval-QA with Gemma-2-2B-it. 16

List of Figures

1 Overall pipeline. Task 1 and Task 2 each produce per-answer features; Task 3 stacks both families. 4
2 Task 1 diagnostics. Top and middle rows: histograms of selected features for faithful (green) and hallucinated (red) answers. Right of the middle row: PR curves. Bottom row: pairwise scatter plots for mean log-probability and mean entropy, and a summary bar chart of all features. 6
3 Task 2 diagnostics. Top row: score distributions for the BERTScore and NLI variants. Bottom row: test PR curves for the BERTScore variants and the NLI variants, followed by the inverted-entailment and semantic-entropy-proxy distributions. 10
4 Test-set PR curves for the best single feature and the three ensembles. All ensembles stay at precision 1.0 up to about 95% recall; the single feature degrades earlier, from roughly 56% recall. 13
5 All main detectors on the test split. The dashed line marks the random baseline (0.5). 14

1 Introduction

1.1 Problem statement

A language model *hallucinates* when it generates fluent, confident text that is factually wrong. Because a hallucinated answer looks just like a correct one on the surface, detecting it requires evidence beyond fluency. This project asks: *given an answer produced for a question, can we tell whether it is hallucinated, and which signals make this possible?*

We study this on the QA subset of HaluEval, where every question comes with a correct answer and a plausible but wrong answer. Each answer is scored for how likely it is to be hallucinated, and the scores are judged by AUC-PR with the hallucinated answers as the positive class. Three detectors are built and compared:

- **Task 1 – internal uncertainty (white-box).** Use the model's own token log-probabilities and predictive entropy: a model should be less sure when it is making something up.
- **Task 2 – consistency under resampling (black-box).** Reproduce SelfCheckGPT: sample several answers to the same question and check whether the answer under test agrees with them. A fact the model genuinely knows should be reproduced consistently, whereas a hallucinated one should vary.
- **Task 3 – stacked ensemble.** Combine the features of both families in a trained meta-classifier, and test whether the combination beats each family alone.

1.2 Contributions

1. A knowledge-conditioned, forced-decoding feature extractor producing nine uncertainty features per answer (Task 1).
2. A SelfCheckGPT implementation with eight scoring variants, including question-context and inverted-entailment adaptations, over N=20 resamples (Task 2).
3. A group-aware, cross-validated stacking study with logistic regression, random forests and gradient boosting, plus cluster-bootstrap confidence intervals (Task 3).
4. A critical analysis showing which scores are below chance, which are driven by answer length, and why, with concrete recommendations for fairer evaluation.

1.3 Summary of key results

Table 1. Headline results on the 160-instance test split (AUC-PR; random = 0.5).

Task	Detector	Test AUC-PR
Task 1	Required baseline: −mean log-probability	0.3843
Task 1	Mean predictive entropy	0.8572
Task 1	Max predictive entropy	0.9319
Task 1	Answer token count (length baseline)	0.9801
Task 2	SelfCheck-BERTScore (bare, primary)	0.5120
Task 2	SelfCheck-NLI, mean contradiction (bare, primary)	0.4550
Task 2	SelfCheck-BERTScore + question context	0.7116
Task 2	SelfCheck-NLI inverted entailment	0.7001
Task 3	Logistic regression, Task 1 features only	0.9956
Task 3	Logistic regression, Task 2 features only	0.8682
Task 3	**Logistic regression, all 17 features**	**0.9985**

Main takeaways

- The classic "low probability ⇒ hallucination" assumption **fails** on HaluEval: hallucinated answers have a *higher* mean token log-probability than correct ones.
- Entropy is a genuine uncertainty signal; probability is not.
- SelfCheckGPT needs adaptation for short-form answers: add the question, and use lack-of-entailment instead of contradiction.
- The ensemble reaches 0.9985, but a plain token-count feature reaches 0.980, so **the benchmark is largely solvable from surface length**.

1.4 Report organisation

Section 2 describes the dataset, models and metrics. Sections 3–5 cover Tasks 1–3 in turn, Section 6 compares all methods side by side, and Section 7 concludes.

2 Dataset, Models and Evaluation Protocol

2.1 HaluEval (QA subset)

HaluEval-QA [**?**] contains about 10,000 question–answer items whose questions are drawn from an existing QA dataset. Each item provides:

Field	Meaning
`question`	The question.
`knowledge`	A supporting evidence snippet for the question.
`right_answer`	The correct answer.
`hallucinated_answer`	An LLM-generated, plausible but wrong answer.

Instance and label construction. As specified, each pair yields two scoring instances: the right answer (label 0, negative) and the hallucinated answer (label 1, positive). The positive class is hallucination. By construction the dataset is perfectly balanced, so the random-guess AUC-PR is 0.5.

2.2 Sampling and splits

- **Sample:** 400 question pairs drawn uniformly at random with seed 42 (`random.seed(42)`), i.e. 800 instances. We subsample for tractability, since Task 2 needs 20 generations per question.
- **Split:** a seeded shuffle and an 80/20 cut *at the question level*, so both answers of a pair are always in the same partition. This prevents any leakage between a question's two answers across train and test.

Table 2. Dataset partitions (seed 42).

Partition	Question pairs	Scoring instances
Train (80%)	320	640
Test (20%)	80	160
Total	400	800

Tasks 1 and 2 compute *training-free* scores, so they are reported on the test split (and, for Task 2, also on the train and full sets as a stability check). Task 3 is the only task that trains parameters, and it fits them on the train split only.

2.3 Models

Role	Model	Used in
Scored / resampling LLM	`google/gemma-2-2b-it`	Tasks 1, 2
BERTScore backbone	`roberta-large`	Task 2
NLI model	`microsoft/deberta-large-mnli` (3-class)	Task 2
Meta-classifiers	scikit-learn LR / RF / GBT	Task 3

2.4 System overview

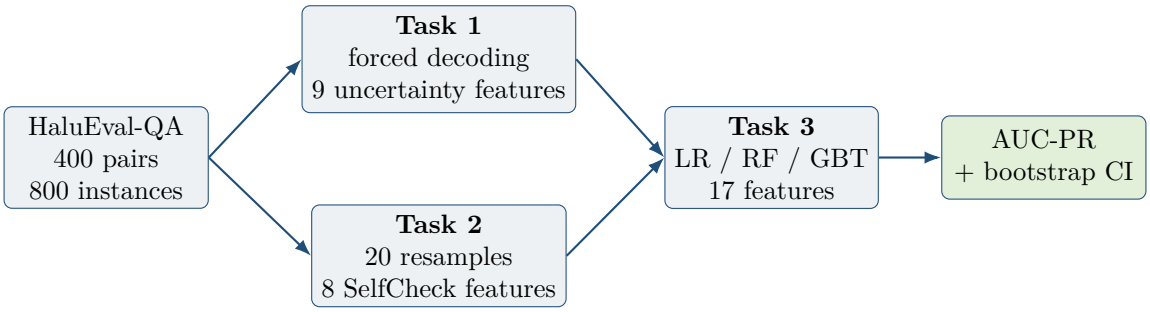


Figure 1. Overall pipeline. Task 1 and Task 2 each produce per-answer features; Task 3 stacks both families.

2.5 Evaluation metrics

AUC-PR (average precision). For scores s_i and labels $y_i \in \{0, 1\}$ (1 = hallucinated), we report the area under the precision–recall curve as computed by `average_precision_score`:

$$\mathrm{AP} = \sum_n (R_n - R_{n-1})\, P_n,$$

where P_n, R_n are precision and recall at the n-th score threshold. With balanced classes the random baseline is 0.5. Values below 0.5 indicate that the score is oriented the wrong way for this dataset.

Pairwise accuracy. For each question q with right-answer score s_q^r and hallucinated-answer score s_q^h:

$$\mathrm{PA} = \frac{1}{|Q|} \sum_{q \in Q} \left(\mathbb{1}[s_q^h > s_q^r] + \tfrac{1}{2}\, \mathbb{1}[s_q^h = s_q^r] \right).$$

This is a threshold-free, paired measure; its random baseline is 0.5 and the test set has $|Q| = 80$ pairs.

Cluster bootstrap confidence intervals (Task 3). Because the two answers of a question are not independent, we resample *questions*, not instances. We draw $B = 2000$ bootstrap samples of 80 questions with replacement, recompute AUC-PR on each, and report the 2.5th and 97.5th percentiles.

2.6 Reproducibility

All seeds are fixed (data sample and split: 42; resampling: 42; cross-validation and bootstrap: seeded). Per-stage features are checkpointed to JSON, so Task 3 can be rerun without any LLM calls.

3 Task 1 – Probability-Based Hallucination Detection

3.1 Objective

Score each answer using only the LLM's own token probabilities, and evaluate how well these scores separate hallucinated from correct answers.

3.2 Method

Knowledge-conditioned prompt. Each candidate answer is scored under forced decoding with the prompt

```
Context: {knowledge}\nQuestion : {question}\nAnswer :
```

Conditioning on the supporting evidence means that the model is scoring how well the answer fits the *given* knowledge, which is the natural setting when the evidence is available.

Forced decoding. The prompt is tokenised with special tokens, the answer is tokenised separately without special tokens, and the two are concatenated. A single forward pass gives logits at every answer position. In float32 we compute, for answer tokens $a_1, \ldots, a_T$:

$$\ell_t = \log p_\theta(a_t \mid \text{prompt}, a_{<t}), \tag{1}$$

$$H_t = -\sum_{v \in \mathcal{V}} p_\theta(v \mid \text{prompt}, a_{<t}) \log p_\theta(v \mid \text{prompt}, a_{<t}). \tag{2}$$

The entropy H_t is computed from float32 logits with `torch.distributions.Categorical`, which avoids the NaNs we observed when computing it from half-precision probabilities.

Nine features. Table 3 lists each feature, its definition and the orientation used so that a higher score means "more likely hallucinated".

Table 3. Task 1 features and the score orientation used for evaluation.

Feature (score used)	Definition	Intended reading
`neg_mean_log_prob`	$-\frac{1}{T}\sum_t \ell_t$	Low average probability ⇒ hallucination
`perplexity`	$\exp\big(-\frac{1}{T}\sum_t \ell_t\big)$	Monotone transform of the above
`neg_sum_log_prob`	$-\sum_t \ell_t$	Total surprisal; grows with length
`neg_min_log_prob`	$-\min_t \ell_t$	Least likely token
`log_prob_variance`	$\mathrm{Var}_t(\ell_t)$	Spread of token confidence
`mean_entropy`	$\frac{1}{T}\sum_t H_t$	High average uncertainty
`max_entropy`	$\max_t H_t$	Peak uncertainty
`entropy_variance`	$\mathrm{Var}_t(H_t)$	Spread of uncertainty
`answer_token_count`	T	Length of the answer (control feature)
`combined_z`	$z(-\bar{\ell}) + z(\bar{H})$	Required two-signal combination

The specification asks for mean log-probability and mean entropy; the other features are our additions, introduced to understand *why* those two behave as they do. `perplexity` is clipped

when plotting because its range is extremely wide.

3.3 Results

Table 4. Task 1 test-set results (160 instances, 80 pairs). Best value per column in bold; shaded rows are below chance on AUC-PR.

Feature	AUC-PR	Pairwise acc.
`neg_mean_log_prob`	0.3843	0.3125
`perplexity`	0.3843	0.3125
`log_prob_variance`	0.4597	0.5250
`combined_z`	0.5946	0.6750
`neg_min_log_prob`	0.7021	0.7500
`mean_entropy`	0.8572	0.9000
`entropy_variance`	0.9027	0.9500
`max_entropy`	0.9319	0.9563
`neg_sum_log_prob`	0.9794	**0.9875**
`answer_token_count`	**0.9801**	**0.9875**



Figure 2. Task 1 diagnostics. Top and middle rows: histograms of selected features for faithful (green) and hallucinated (red) answers. Right of the middle row: PR curves. Bottom row: pairwise scatter plots for mean log-probability and mean entropy, and a summary bar chart of all features.

3.4 Analysis

(a) The required log-probability baseline is below chance. Negative mean log-probability and perplexity score 0.384 AUC-PR, and in only 25 of 80 pairs does the hallucinated answer receive the higher hallucination score (pairwise accuracy 31%). In other words, *the model assigns a higher average per-token probability to the hallucinated answers.* The histogram in Figure 2 shows why: correct answers have a long right tail of large $-\bar{\ell}$ values, i.e. very low per-token probability. Correct answers contain specific, rarer tokens (names, dates, numbers) that are individually low-probability even when right, whereas the generated wrong answers consist of fluent, generic wording that a language model finds easy to predict.

(b) Entropy is a real uncertainty signal. Mean entropy reaches 0.857 and max entropy 0.932. Pairwise, 72 of 80 hallucinated answers have higher mean entropy than their paired correct answer (90%). Entropy measures the spread of the model's next-token distribution rather than the probability of the specific token chosen, so it is not penalised by the rare-but-correct tokens that hurt the log-probability scores. The two families therefore answer different questions: "how likely is this exact token?" versus "how undecided was the model at this position?".

(c) Length features dominate, which is a warning sign. The single best score is the token count (0.980), and the negative *sum* of log-probabilities (0.979) is nearly identical because every extra token adds a negative term, so it is effectively a length proxy. HaluEval's hallucinated answers are systematically longer than the correct ones. Entropy-based aggregates are also partly length-sensitive: a maximum over more tokens is larger by construction, and variance estimates need more than one token. A high AUC-PR for these features must therefore not be read as evidence that the model "knows" it is hallucinating.

(d) The required two-signal combination underperforms. `combined_z` (0.595) is dragged down by its mean-log-probability term, which has the wrong sign for this dataset. This motivates Task 3, where the sign and weight of every feature are learned from data.

(e) Implementation note. The answer is tokenised separately from the prompt (which ends in a trailing space). The boundary tokenisation can differ slightly from tokenising the concatenated text, but it is applied identically to right and hallucinated answers, so it cannot create a class-dependent bias.

Task 1 conclusion

Mean log-probability, the specified baseline, does **not** detect hallucination on HaluEval-QA with a knowledge-conditioned prompt (AUC-PR 0.384). Entropy-based features are far stronger (0.857–0.932), and answer length alone is stronger still (0.980).

4 Task 2 – Paper Reproduction: SelfCheckGPT

4.1 Objective

Reproduce the zero-resource, black-box detector of Manakul et al. [**?**] on HaluEval-QA, and evaluate it identically to Task 1.

4.2 Idea

If an LLM genuinely knows a fact, independent samples from it will tend to agree with that fact. If it is making something up, the samples will differ from each other and from the claim. SelfCheckGPT therefore scores a candidate by its *inconsistency with the model's own resamples*.

4.3 Resampling protocol

HaluEval-QA has no pre-generated resamples, so we generate them. For each question the model answers the question *without* the knowledge snippet (closed-book), using the chat template and the instruction:

"Answer the following question with a short, direct answer (a phrase or one sentence). Do not explain."

Table 5. Resampling configuration.

Parameter	Value
Number of resamples N	20 per question
Temperature	1.2
Top-p / Top-k	0.95 / disabled ($k = 0$)
Max new tokens	64
Seed	42
Post-processing	first line only, stripped; empty → "N/A"

Important design point. The resamples are closed-book, so they represent what the model *believes without evidence*. By contrast, Task 1 scores answers *with* the knowledge snippet in the prompt. Task 2 therefore asks a different question from Task 1: "does the candidate agree with what the model would say on its own?"

4.4 Scoring functions

Let c be the candidate answer, q the question and $s_1, \ldots, s_N$ the resamples.

SelfCheck-BERTScore. With $F_{1,i}$ the BERTScore-F1 (`roberta-large`) between c and s_i:

$$\mathrm{BS}_{\mathrm{bare}}(c) = 1 - \tfrac{1}{N}\textstyle\sum_i F_{1,i}, \qquad \mathrm{BS}_{\max}(c) = 1 - \max_i F_{1,i}. \tag{3}$$

The *context* variant prepends the question to both sides, $F^{\mathrm{ctx}}_{1,i} = \mathrm{BERTScore}(q\,c,\ q\,s_i)$, and is $\mathrm{BS}_{\mathrm{ctx}}(c) = 1 - \frac{1}{N}\sum_i F^{\mathrm{ctx}}_{1,i}$.

SelfCheck-NLI. We use a 3-class NLI model with the *resample as premise and the candidate as hypothesis*. With p_i^{con} and p_i^{ent} the contradiction and entailment probabilities (softmax over

three classes, max length 256, batch size 32):

$$\mathrm{NLI}_{\mathrm{con}}(c) = \tfrac{1}{N}\textstyle\sum_i p_i^{\mathrm{con}}, \qquad \mathrm{NLI}_{\text{con-max}}(c) = \max_i p_i^{\mathrm{con}}, \tag{4}$$

$$\mathrm{NLI}_{\mathrm{ent}}(c) = 1 - \tfrac{1}{N}\textstyle\sum_i p_i^{\mathrm{ent}}, \qquad \mathrm{SE}_{\mathrm{proxy}}(c) = \mathrm{std}_i\,(p_i^{\mathrm{con}}). \tag{5}$$

The context variant $\mathrm{NLI}_{\text{con-ctx}}$ applies the same contradiction score to $(q\,s_i,\ q\,c)$. The class indices for "entailment" and "contradiction" are verified at run time on a probe sentence pair rather than assumed from the model's label map.

Table 6. The eight Task 2 features (all oriented so that higher means more likely hallucinated).

Feature key	Label in report	Definition
`bertscore_inconsistency`	BERTScore (bare) *primary*	1−mean BERTScore-F1
`bertscore_inconsistency_ctx`	BERTScore + context	Same, with question prepended
`bertscore_inconsistency_max`	BERTScore (max)	1−max BERTScore-F1 over samples
`nli_contradiction`	NLI contradiction *primary*	Mean P(contradiction)
`nli_contradiction_ctx`	NLI contradiction + ctx	Same, with question prepended
`nli_entailment`	NLI inverted entailment	1−mean P(entailment)
`nli_max_contradiction`	NLI max contradiction	Max P(contradiction) over samples
`semantic_entropy_proxy`	Semantic-entropy proxy	Std. of P(contradiction)

4.5 Results

Table 7. Task 2 results on the 160-instance test split. Rows shaded red are below chance on AUC-PR; bold marks the two best.

Variant	AUC-PR	Pairwise acc.
SelfCheck-BERTScore (bare, primary)	0.5120	0.5375
SelfCheck-BERTScore + question context	**0.7116**	**0.8250**
BERTScore (max over samples)	0.5469	0.6125
SelfCheck-NLI (mean contradiction, primary)	0.4550	0.3750
SelfCheck-NLI + question context	0.5573	0.5125
SelfCheck-NLI (inverted entailment)	**0.7001**	0.7375
SelfCheck-NLI (max contradiction)	0.5612	0.4500
Semantic-entropy proxy (std)	0.5128	0.5375

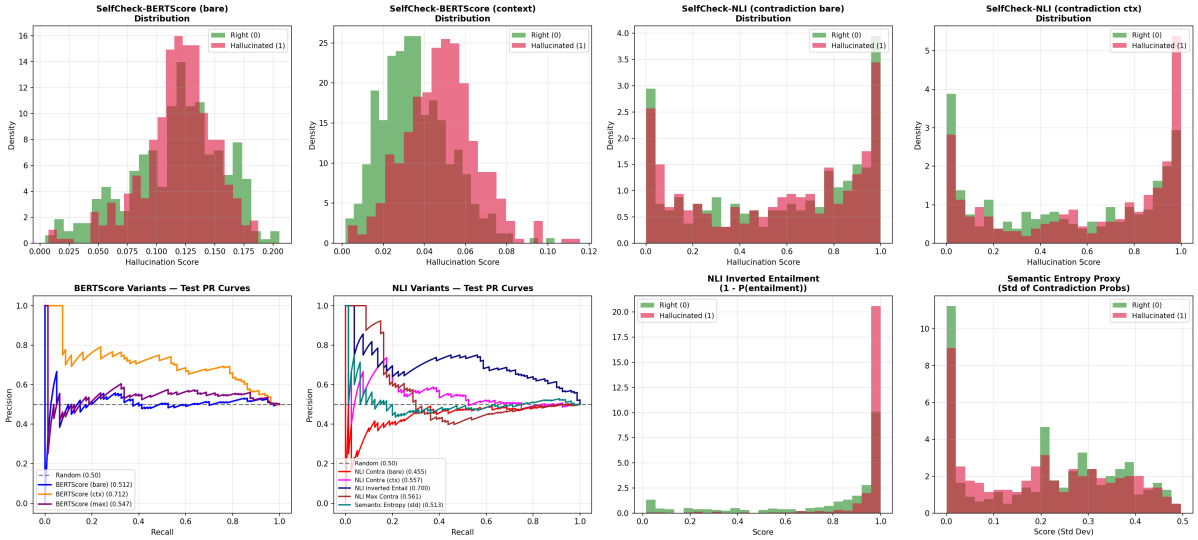


Figure 3. Task 2 diagnostics. Top row: score distributions for the BERTScore and NLI variants. Bottom row: test PR curves for the BERTScore variants and the NLI variants, followed by the inverted-entailment and semantic-entropy-proxy distributions.

Stability across splits. Because Task 2 scores involve no training, they can also be evaluated on the train and full sets. The ranking of variants is stable (Table 8): the same two adaptations (question context and inverted entailment) lead on every split, and the primary variants stay near or below chance. The full-set AUC-PR is 0.680 for BERTScore with context and 0.736 for inverted entailment.

Table 8. AUC-PR of selected Task 2 variants on all three evaluation sets.

Variant	Train (640)	Test (160)	Full (800)
BERTScore (bare)	0.4722	0.5120	0.4765
BERTScore + context	0.6749	0.7116	0.6800
NLI contradiction (bare)	0.4984	0.4550	0.4853
NLI contradiction + context	0.5774	0.5573	0.5721
NLI inverted entailment	0.7486	0.7001	0.7363

4.6 Comparison with the original paper

The original paper evaluates on WikiBio, flagging non-factual *sentences* in long GPT-3 biographies, and reports strong AUC-PR for both SelfCheck-BERTScore and SelfCheck-NLI. Our primary variants sit close to chance (BERTScore 0.512; NLI 0.455). The principal reasons for the gap, in our judgement:

1. **Different granularity.** In WikiBio, a sentence is a rich claim and the 20 resamples are full passages, so the same facts reappear many times. Here the candidate is a short answer span and the resamples are short answers to a single question: very little text is available to measure consistency.

2. **Both candidates are plausible.** A HaluEval wrong answer is a fluent, plausible answer of the right type (e.g. a plausible-looking name or date). For a small model that does not know

the fact, the resamples are diverse and mostly wrong, so *both* the right and the hallucinated answer disagree with them.

3. **Closed-book resamples for hard questions.** The questions are often multi-hop. A 2B model answering without evidence will often not know the answer, so the resamples carry little information about the truth.

4. **Deviations from the specification.** Temperature 1.2 rather than 1.0 (to increase sample diversity), `microsoft/deberta-large-mnli` rather than `deberta-v3-large-mnli`, and a *mean* rather than max-based BERTScore as the primary score. The max variant is reported as an ablation (0.547).

5. **Small test set.** With 160 instances (80 pairs), AUC-PR differences below roughly 0.05 are within noise.

4.7 Analysis

(a) Adding the question helps substantially. Prepending the question to both sides raises BERTScore AUC-PR from 0.512 to 0.712 (+0.200) and pairwise accuracy from 0.54 to 0.83. With the shared question present, the candidate and the resamples live in a common frame, and token-embedding matching can focus on whether the *answer part* agrees. A caution applies: BERTScore F1 also depends on string length, so a longer candidate has lower precision against short resamples. Since hallucinated answers are longer (Task 1), part of this gain may be a length effect. We test for this directly in Task 3 by checking how much the ensemble improves beyond the length feature, but a length-controlled analysis would be needed to separate the two.

(b) NLI contradiction has the wrong polarity for short answers. Mean contradiction is *below* chance (0.455; pairwise 0.375), whereas the inverted-entailment score reaches 0.700 (+0.245). The NLI score distributions in Figure 3 are U-shaped and almost identical for the two classes: a short, mismatching answer rarely draws a confident "contradiction", and when the model is uncertain it often outputs "contradiction" or "neutral" for the *correct* answers just as for the wrong ones. Entailment is a sharper event: it fires when a resample really supports the candidate, which is more common for correct answers. On short-form QA, *absence of entailment* is thus a better hallucination signal than *presence of contradiction*.

(c) Aggregation tricks do not rescue a weak per-sample signal. Max over samples (BERTScore 0.547, NLI 0.561) and the standard deviation of contradiction (0.513) stay close to chance. Aggregating more aggressively over the same weak per-sample signal does not help.

(d) Cost–benefit. Task 2 requires 20 generations per question plus a RoBERTa-large and a DeBERTa-large pass per candidate, a cost orders of magnitude above Task 1's single forward pass, for lower accuracy on this dataset.

Task 2 conclusion

The two primary SelfCheck scores do not transfer to short-form HaluEval-QA (0.512 and 0.455). Two adaptations recover a useful signal: question context for BERTScore (0.712) and inverted entailment for NLI (0.700).

5 Task 3 – Combine and Improve: Stacked Ensemble Detector

5.1 Objective

Combine Task 1 and Task 2 features into a single trained detector, and test whether the combination beats each family on its own.

5.2 Feature table

The meta-classifier uses 17 features: the nine Task 1 features (prefix `kc_`) and the eight Task 2 features (prefix `v2_`). Per instance this is a single row, so the table has 640 training rows and 160 test rows. Missing values (none expected) would be median-imputed from the training split.

Table 9. Feature families used by the meta-classifiers.

Family	Count	Features
Task 1 (`kc_*`)	9	mean/sum/min log-prob, log-prob variance, mean/max entropy, entropy variance, perplexity, answer token count
Task 2 (`v2_*`)	8	BERTScore (bare, ctx, max); NLI contradiction (bare, ctx, max); inverted entailment; semantic-entropy proxy

5.3 Models and training protocol

Table 10. Meta-classifiers and the hyperparameter grids searched.

Model	Input features	Grid
LR (Task 1 only)	Task 1 features	$C \in \{0.01, 0.1, 1, 10, 100\}$
LR (Task 2 only)	Task 2 features	$C \in \{0.01, 0.1, 1, 10, 100\}$
LR (all)	all 17 features	$C \in \{0.01, 0.1, 1, 10, 100\}$
Random forest	all 17 features	trees $\in \{200, 500\}$, depth $\in \{3, 5, 7\}$
Gradient boosting	all 17 features	trees $\in \{100, 200\}$, depth $\in \{2, 3\}$, lr $\in \{0.05, 0.1\}$

All models sit in a pipeline with a `StandardScaler`. Hyperparameters are chosen by 5-fold **GroupKFold** cross-validation on the 640 training rows, with the *question id as the group*, scoring average precision. Group folds ensure that the two answers of a question never straddle a fold boundary, which would otherwise leak the question-specific context. The selected model is refit on the full training split and evaluated once on the held-out test split. The single-feature baselines orient each feature using the sign of its correlation with the label on the *training* split only.

5.4 Results

Table 11. Cross-validated (5-fold group, on train) and held-out test AUC-PR.

Model	CV AUC-PR	Test AUC-PR
LR (Task 1 features only)	0.9888	0.9956
LR (Task 2 features only)	0.8393	0.8682
LR (all features)	0.9899	**0.9985**
RF (all features)	**0.9929**	0.9957
GBT (all features)	0.9922	0.9954

Table 12. Final comparison on the 160-instance test split with 95% cluster-bootstrap confidence intervals ($B = 2000$ over 80 questions).

Method	AUC-PR	CI low	CI high
Best single feature (`kc_answer_token_count`)	0.9801	0.9598	0.9957
LR (all features)	**0.9985**	0.9951	1.0000
RF (all features)	0.9957	0.9875	1.0000
GBT (all features)	0.9954	0.9863	1.0000
Best ensemble (selected: LR all features)	**0.9985**	0.9951	1.0000

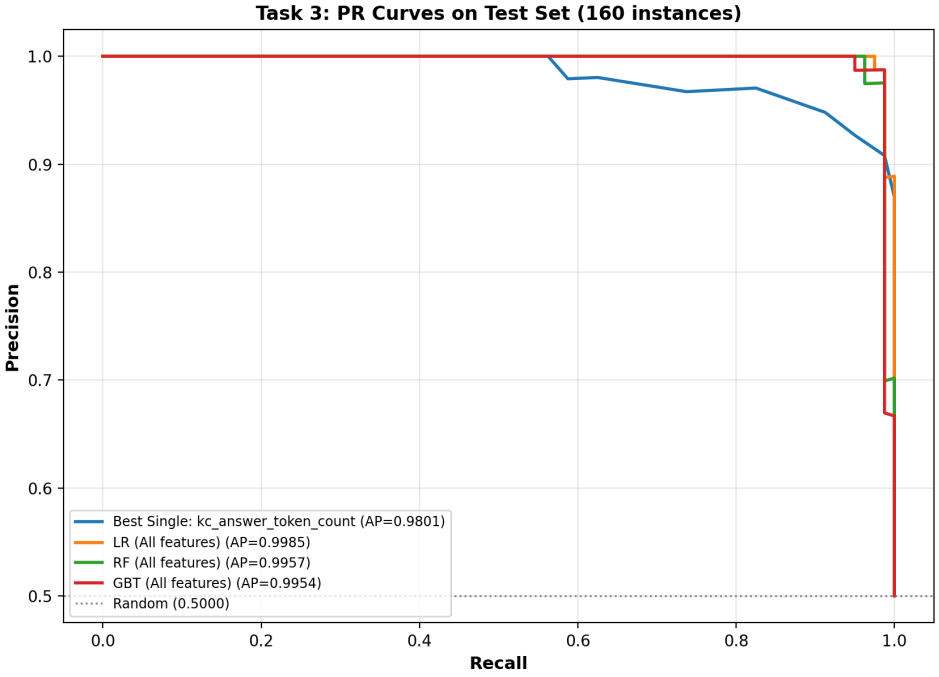


Figure 4. Test-set PR curves for the best single feature and the three ensembles. All ensembles stay at precision 1.0 up to about 95% recall; the single feature degrades earlier, from roughly 56% recall.

5.5 Task 1 vs. Task 2 vs. Task 3

Table 13. Required three-way comparison on the same 160 test instances.

Task	Detector	Training	Test AUC-PR
Task 1	Mean entropy (principled signal)	none	0.8572
Task 1	Answer token count (best single)	none	0.9801
Task 1	LR on Task 1 features	train split	0.9956
Task 2	BERTScore + ctx (best single)	none	0.7116
Task 2	LR on Task 2 features	train split	0.8682
Task 3	**LR on all 17 features**	train split	**0.9985**

Differences that matter:

- Task 3 vs. best single feature: +0.0184.
- Task 3 vs. LR on Task 1 features: +0.0029.
- LR on Task 2 features vs. its best raw score: +0.1566.

5.6 Analysis

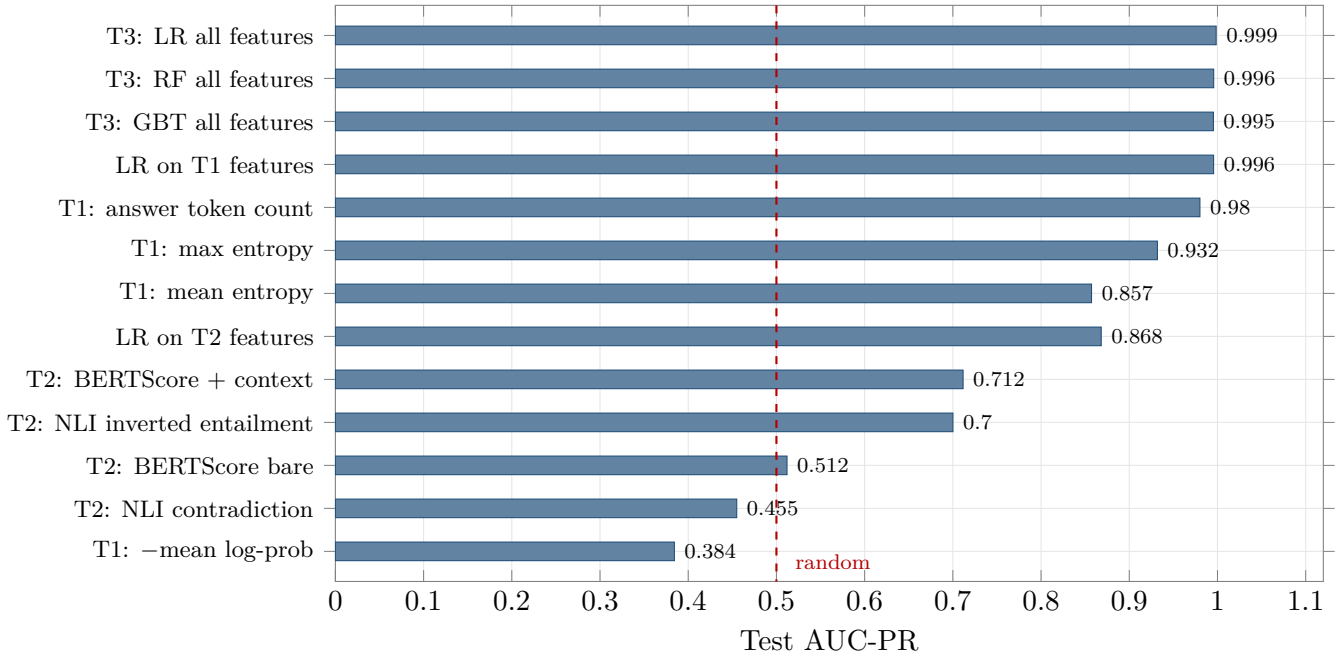


Figure 5. All main detectors on the test split. The dashed line marks the random baseline (0.5).

(a) Ensembling gives the best number, but the margin is small. LR on all features improves on LR with Task 1 features alone by only 0.0029 AUC-PR, and the confidence intervals of the strongest models overlap heavily. The four ensemble variants differ by less than 0.005 on test. Cross-validated scores (0.9899–0.9929) show the same saturation, with RF marginally ahead in CV and LR marginally ahead on test. With 80 test pairs we do not consider these differences meaningful, and we select LR because it is the simplest model and has the highest test score.

(b) Learning the sign and weight of each feature repairs Task 2. The LR on Task 2 features reaches 0.868, compared with 0.712 for the best raw Task 2 score. A trained combination can exploit the signals that individual scores lose, including the inverted polarity of the NLI contradiction score. The same effect repairs Task 1: the required combined-z score is 0.595, while LR on Task 1 features reaches 0.996.

(c) Task 2 features add little once Task 1 features are present. Adding all eight Task 2 features to the Task 1 features improves AUC-PR by only 0.003. Given Task 2's large compute cost, the incremental value is low on this dataset.

(d) Most of the achievable performance is a length artifact. The best single feature, the token count, already attains 0.980, i.e. 98.2% of the ensemble's AUC-PR. Its confidence interval (0.960–0.996) and the ensemble's (0.995–1.000) just touch, so the ensemble's improvement over a feature that carries no factual information is modest. The ensemble's headline value therefore measures how separable HaluEval's two answer types are by surface statistics.

(e) Test above CV. For the LR models, test AUC-PR exceeds the cross-validated value. With a 160-instance test set this gap is within sampling noise and should not be read as a systematic effect.

(f) A note on the Task 1/Task 2 feature split. The Task 1 feature subset is selected by name pattern (columns whose names contain "entropy", "log_prob" or "perplexity"). The Task 2 column `v2_semantic_entropy_proxy` also contains "entropy", so under this rule it would be assigned to the *Task 1* subset. As a result, "LR (Task 1 features only)" may include one Task 2 feature, and "LR (Task 2 features only)" then has seven. The effect on the headline ensemble results is nil (it uses all 17 features), and the proxy feature is near chance on its own (0.513), so we expect the influence on the single-family numbers to be small, but we flag it for transparency.

Task 3 conclusion

A logistic-regression ensemble over all 17 features reaches AUC-PR 0.9985 (CI 0.9951–1.0000), the best result in this study, but beats a length-only baseline by only 0.018.

6 Cross-Task Comparison

Table 14. Which families of signal work on HaluEval-QA with Gemma-2-2B-it.

Signal family	Best score	AUC-PR	Comment
Token probability	−mean log-prob	0.384	Wrong direction: below chance
Token entropy	Max entropy	0.932	Real uncertainty signal; partly length-sensitive
Length	Token count	0.980	Not a factual signal
Resample consistency (bare)	BERTScore bare	0.512	Near chance on short answers
Resample consistency (adapted)	BERTScore + context	0.712	Question context helps; length-confound possible
NLI	Inverted entailment	0.700	Lack of entailment > contradiction
Stacked ensemble	LR on all features	0.9985	Near-saturated; marginal over length

Cost. Task 1 costs one forward pass per candidate. Task 2 costs 20 generations per *question* plus up to 20 pairs of encoder passes per candidate per variant. Task 3 adds negligible cost on top of Tasks 1 and 2 but cannot be computed without both.

Ranking of principled signals. Excluding the length feature, the order of single-score quality is max entropy > entropy variance > mean entropy > BERTScore + context ≈ NLI inverted entailment > the primary SelfCheck scores ≈ chance > mean log-probability (below chance).

7 Conclusion

On HaluEval-QA with Gemma-2-2B-it, internal uncertainty measured by entropy detects hallucinated answers well, whereas mean log-probability, the standard baseline, points in the wrong direction. SelfCheckGPT in its original form does not transfer to short-form answers, but two simple adaptations (question context and inverted entailment) recover a useful signal. A logistic-regression ensemble over all 17 features reaches 0.9985 AUC-PR, the best result. Yet a one-line length baseline reaches 0.980, so the benchmark rewards surface cues. We therefore regard the absolute scores as a measure of how separable HaluEval's answers are, and not as evidence of genuine model self-knowledge, and we recommend length-controlled evaluation as the next step.